

UNIVERSIDAD NACIONAL AUTONOMA DE MEXICO

FACULTAD DE INGENIERIA

BASES DE DATOS

ALUMNO: LARA CRUZ RAUL

PROFESOR: ING. ARREOLA FRANCO FERNANDO

TAREA: 3

GRUPO: 01

Investigar de la interfaz de PostgreSQL

- Concepto de roles y privilegios
- Como dar y quitar privilegios
- privilegios a nivel sistema y objeto
- Diferencia entre rol y usuario

1. Concepto de roles y privilegios

¿Qué es un rol?

En PostgreSQL, un **rol** es una entidad que sirve para administrar la seguridad y los permisos de la base de datos.

Un rol puede representar:

- Un usuario individual.
- Un grupo de usuarios.
- Una combinación de ambos.

PostgreSQL unificó los conceptos de usuarios y grupos en el concepto de **rol**. Por eso, técnicamente, un usuario de PostgreSQL también es un rol configurado con permiso para iniciar sesión.

Por ejemplo:

```
CREATE ROLE alumno LOGIN PASSWORD '12345';
```

Aquí se crea un rol llamado alumno que puede iniciar sesión.

También podemos crear un rol que funcione como grupo:

```
CREATE ROLE profesores;
```

Después podemos agregar usuarios a ese rol.

¿Qué son los privilegios?

Los privilegios son permisos que indican qué acciones puede realizar un rol dentro de PostgreSQL.

Algunos de los más importantes son:

Privilegio	Función
SELECT	Consultar información
INSERT	Insertar registros
UPDATE	Modificar registros
DELETE	Eliminar registros
TRUNCATE	Vaciar una tabla
REFERENCES	Crear referencias mediante claves foráneas
TRIGGER	Crear triggers
CREATE	Crear objetos
CONNECT	Conectarse a una base de datos
EXECUTE	Ejecutar funciones o procedimientos
USAGE	Utilizar determinados objetos, como esquemas

Los privilegios disponibles dependen del tipo de objeto al que se apliquen.

2. ¿Cómo dar y quitar privilegios?

En PostgreSQL se utilizan principalmente dos comandos:

- GRANT → **otorga permisos**.
- REVOKE → **quita permisos**.

GRANT

Por ejemplo, tenemos una tabla llamada `alumnos` y queremos permitir que el usuario `juan` pueda consultarla:

```
GRANT SELECT ON alumnos TO juan;
```

Ahora `juan` puede realizar:

```
SELECT * FROM alumnos;
```

pero no necesariamente podrá modificar los datos.

También podemos otorgar varios privilegios:

```
GRANT SELECT, INSERT, UPDATE ON alumnos TO juan;
```

Esto permite que juan consulte, inserte y modifique registros.

También existe:

```
GRANT ALL PRIVILEGES ON alumnos TO juan;
```

que otorga todos los privilegios aplicables a ese objeto, siempre que quien ejecuta el comando tenga autoridad para concederlos.

Quitar privilegios con REVOKE

Para quitar el permiso de modificación:

```
REVOKE UPDATE ON alumnos FROM juan;
```

Y para quitar todos los privilegios:

```
REVOKE ALL PRIVILEGES ON alumnos FROM juan;
```

PostgreSQL también permite utilizar PUBLIC para otorgar un privilegio a todos los roles:

```
GRANT SELECT ON alumnos TO PUBLIC;
```

Esto significa que cualquier rol podrá consultar la tabla.

3. Privilegios a nivel sistema y a nivel objeto

Aquí hay una diferencia importante.

Privilegios a nivel sistema

Son permisos relacionados con las capacidades generales del rol dentro del servidor o del sistema de PostgreSQL, no solamente con una tabla específica.

Algunos ejemplos son:

- Poder iniciar sesión (LOGIN).
- Crear bases de datos (CREATEDB).
- Crear otros roles (CREATEROLE).
- Ser superusuario (SUPERUSER).
- Poder realizar determinadas operaciones administrativas.
- Poder realizar replicación (REPLICATION).

Por ejemplo:

```
CREATE ROLE administrador  
LOGIN  
CREATEDB  
CREATEROLE  
PASSWORD '12345';
```

Este rol tendría permiso para iniciar sesión, crear bases de datos y crear roles.

En pgAdmin estas opciones aparecen en la pestaña **Privileges** al crear o modificar un rol. Entre ellas están Can login?, Superuser, Create roles?, Create databases, etc.

Privilegios a nivel objeto

Son permisos que se aplican específicamente a un objeto de la base de datos.

Por ejemplo:

- Una tabla.
- Una vista.
- Una secuencia.
- Una función.
- Un esquema.

- Una base de datos.

Por ejemplo:

```
GRANT SELECT ON alumnos TO juan;
```

El permiso SELECT solamente se está otorgando sobre la tabla alumnos.

Otro ejemplo:

```
GRANT INSERT, UPDATE ON alumnos TO juan;
```

juan podrá insertar y modificar información de esa tabla.

La documentación de PostgreSQL establece que los privilegios se pueden aplicar a diferentes objetos y que el propietario de un objeto normalmente tiene todos los privilegios sobre él.

Diferencia rápida

Nivel sistema	Nivel objeto
Afecta capacidades generales del rol	Afecta objetos específicos
LOGIN	SELECT
CREATEDB	INSERT
CREATEROLE	UPDATE
SUPERUSER	DELETE
REPLICATION	EXECUTE
Se relaciona con el servidor/rol	Se relaciona con tablas, vistas, funciones, etc.

4. Diferencia entre rol y usuario

Esta parte es importante porque PostgreSQL no los considera dos entidades completamente diferentes.

En versiones modernas de PostgreSQL, el concepto de usuario está incluido dentro del concepto de rol. Un rol puede configurarse para iniciar sesión y entonces funciona como un usuario.

Por ejemplo:

```
CREATE ROLE juan LOGIN PASSWORD '12345';
```

juan es un **rol**, pero como tiene LOGIN, puede utilizarse como usuario para conectarse a PostgreSQL.

Mientras tanto:

```
CREATE ROLE alumnos;
```

puede utilizarse como un rol de grupo para reunir permisos.

Por ejemplo:

```
GRANT SELECT ON alumnos TO alumnos;
```

Después:

```
GRANT alumnos TO juan;
```

Ahora juan puede obtener los privilegios que tiene el rol alumnos.

En conclusión

Rol = concepto general para administrar identidades y permisos.

Usuario = un rol que normalmente tiene habilitado LOGIN para poder conectarse.

La propia documentación de PostgreSQL señala que cualquier rol puede funcionar como usuario, grupo o ambos.

Fuentes consultadas

PostgreSQL Global Development Group. (2026). *Database roles*. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/user-manag.html>

PostgreSQL Global Development Group. (2026). *Privileges*. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/ddl-priv.html>

PostgreSQL Global Development Group. (2026). *GRANT — define access privileges*. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/sql-grant.html>

PostgreSQL Global Development Group. (2026). *REVOKE — remove access privileges*. PostgreSQL Documentation. <https://www.postgresql.org/docs/current/sql-revoke.html>